Student Number: ST10502961
Surname: Nembambula
Name: Morongwe Moipoledi
Module Code: PDAN8412
Module Name: Programming for Data Analytics II

Part 2: Logistic Regression from Scratch

Part 2: Logistic Regression from Scratch..... 1
Introduction..... 4
1. Dataset selection and justification..... 4
2. Exploratory Data Analysis (EDA)..... 5
2.1 Loading and inspecting the dataset..... 6
figure 1..... 6
2.2 Checking missing values and repeated records..... 6
2.3 Applying data cleaning decisions..... 7
2.4 Creating the bestseller target and examining class distribution..... 7
figure 2..... 8
2.5 Examining numerical distributions and potential outliers..... 8
2.6 Visualizing review and category distributions..... 9
figure 3..... 9
figure 4..... 10
2.7 Exploring title characteristics and subject categories..... 10
figure 5..... 11
3. Feature Selection and Data Preparation..... 11
3.1 Identifying candidate predictors..... 11
3.2 Partitioning before statistical feature selection..... 12
3.3 Checking predictor completeness and variation..... 12
3.4 Removing highly correlated predictors..... 13
3.5 Final feature set..... 13
3.6 Standardising the selected predictors..... 13
4. Model Implementation and Training..... 14
4.1 Implementing Logistic Regression from scratch..... 14
4.2 Defining the training objective..... 14
4.3 Checking the implementation..... 14
4.4 Training the initial model..... 15
figure 6..... 15
4.5 Assessing initial validation performance..... 15
4.6 Retraining with alternative parameters..... 16
4.7 Selecting the classification threshold..... 16
5. Model Interpretation and Evaluation..... 17
5.1 Selecting appropriate evaluate metrics..... 17
5.2 Establishing a baseline..... 17
5.3 Using validation data for modeling decisions..... 18
5.4 Evaluating the frozen model on test data..... 18
5.5 Interpreting the confusion matrix..... 18
5.6 Examining the precision recall curve..... 19
figure7..... 20
5.7 Interpreting the coefficients..... 20
5.8 Assessing reliability and limitations..... 21
6. Conclusion..... 21
References..... 22

Introduction

Knowing which books are likely to sell well can help a publishing company make better choices about which books to acquire, how to market them and where to put its resources. This project looks at whether logistic regression can sort books into bestsellers and non-bestsellers using the book metadata that is available.

The study uses a sample of 20,000 book records taken from the Amazon product metadata dataset hosted by Stanford SNAP. The original data was collected in 2006 and includes titles, sales ranks, category information and review summaries (Stanford SNAP, n.d.). For this analysis, a bestseller is any book with an Amazon sales rank between 1 and 10,000, inclusive. This is a definition made for this project and is not the same as being on an official bestseller list.

The analysis was done in Jupyter Notebook. Apache Spark was used to explore and clean the data and to create the features. The logistic regression was written from scratch in NumPy, following the mathematical principles in James et al. (2023). I compared several training configurations and then tested the chosen model on a separate test set. I used metrics that suit an imbalanced classification problem.

This report covers the dataset assessment, how the data was prepared, the exploratory findings, the model development and the evaluation results. It also discusses what the findings mean for the publishing company and the limitations of the study. Because the data is a snapshot from the past, the project tests how well we can classify books that were already bestsellers. It cannot show whether a book that hasn't been published yet will be a commercial success in the future.

1. Dataset selection and justification

I chose the Amazon product metadata dataset hosted by Stanford SNAP for this project. The full collection has 393,561 book records and includes product identifiers, titles, sales ranks, category memberships and review information (Stanford SNAP, n.d.). I drew a reproducible random sample of 20,000 books, using random seed 42, from the records that had a valid identifier, a title and a positive sales rank. This is more than the assignment's minimum of 10,000 records, and it still keeps the processing manageable.

I picked this dataset because its sales rank field gives a sales-related way to define the classification target. Books ranked between 1 and 10,000, inclusive, were labelled as bestsellers, and books ranked above 10,000 were placed in the other class. This is a working definition for the project. Sales rank does not show how many copies were sold, and it does not mean a book is on an official bestseller list.

Quality Criterion	Assessment and Justification
Relevance	Sales rank gives us a sales-related outcome to predict, while titles and category information supply the candidate predictors. Review information was useful for exploring the data, but I left it out of the main model because it isn't available before a book is published.
Source credibility	Stanford SNAP documents how the dataset is structured and links to the academic research behind it, which makes the data easier to trace back to its source (Stanford SNAP, n.d.; Leskovec, Adamic and Huberman, 2007).
Completeness	Inspection identified 901 missing category paths. These were labelled "Unknown" and flagged separately. Additionally, 5,756 books without reviews had zero ratings, which were converted to null values to distinguish unavailable ratings from poor satisfaction.
Validity	Checks found no non-positive sales ranks, negative category or review counts, or ratings outside the range of 0–5.
Repeated titles	Seventy-five normalized titles appeared more than once. These records were retained because they may represent different editions or unrelated books sharing a title. Identical normalized titles were grouped during partitioning to reduce potential leakage.
Class balance	Only 281 books met the bestseller definition. This imbalance required evaluation using precision, recall, F1 and average precision rather than accuracy alone.
Timeliness and coverage	The data were collected in 2006 and reflect an Amazon catalogue snapshot. They may not represent current publishing conditions, other sales channels or the client's market.

Overall, the dataset works well for showing how to do analysis in Spark and how to build logistic regression from scratch. It is much less suitable for forecasting future bestsellers. It has no predictors that are confirmed to be available before publication, no later sales outcomes, and none of the important commercial variables, such as price, marketing spend and an author's past sales. The results therefore only show how well we can classify books by their past sales rank, and they should not be treated as proven predictions of future commercial success.

2. Exploratory Data Analysis (EDA)

I did the exploratory data analysis in Jupyter Notebook using Apache Spark, with the aim of understanding the dataset, checking its quality and finding patterns that matter for modelling. Spark calculated the summaries and frequencies, and Matplotlib was used to display the charts.

2.1 Loading and inspecting the dataset

The first thing that I did was loading the CSV file into a Spark DataFrame using an explicit schema. Identifiers and text fields were stored as strings, count variables as integers and average ratings as decimal values. Keeping the ASIN as text meant that any leading zeros were not lost.

The imported dataset had 20,000 records and seven columns, which meets the assignment's minimum size requirement. I checked the column names, the data types and the first five records to make sure the file had loaded as expected. The data comes from the Amazon product metadata collection hosted by Stanford SNAP (Stanford SNAP, n.d.). Figure 1 below shows the first 5 rows and the structure of the dataset.

```
Number of records: 20000
Number of columns: 7

Column names:
['asin', 'title', 'salesrank', 'category_count', 'review_count', 'average_rating', 'category_paths']

Dataset schema:
root
 |-- asin: string (nullable = true)
 |-- title: string (nullable = true)
 |-- salesrank: long (nullable = true)
 |-- category_count: integer (nullable = true)
 |-- review_count: long (nullable = true)
 |-- average_rating: double (nullable = true)
 |-- category_paths: string (nullable = true)

+----------+--------------------------------------------------+---------+--------------+------------+--------------+
|      asin|                                             title|salesrank|category_count|review_count|average_rating|
+----------+--------------------------------------------------+---------+--------------+------------+--------------+
|0001057170|                      CLASSIC CONNOLLY BOXED SET|   708137|             3|           2|           3.5|
|0001360841|           Bumpy Dog Helps Out (Toy Town Stories)|  3442302|             0|           0|           0.0|
|0002189615|                                            Foggy|   602839|             4|           3|           4.5|
|0002200295|Butterflies Amd Moths of Britain and Europe (Co...|  2793685|             2|           0|           0.0|
|0002251493|       It's a Dog's Life : Journal for Your Pet, A|  1265855|             1|           1|           4.0|
+----------+--------------------------------------------------+---------+--------------+------------+--------------+
only showing top 5 rows
```

figure 1

2.2 Checking missing values and repeated records

I counted null values in every column and also checked separately for blank text values. The null check found 901 missing category paths, which is about 4.51% of the dataset. None of the other six original columns had any nulls.

I also looked for completely duplicate rows, repeated ASINs and repeated normalised titles. There were no duplicates. Normalising the titles meant removing differences in capitalisation and spacing. Seventy-five distinct normalised titles appeared more than once. I did not remove these automatically, because they could be different editions of the same book or unrelated books that happen to share a title.

The numerical validity checks found no sales ranks that were zero or negative, no negative category or review counts, and no ratings outside the 0–5 range. A closer look showed that

5,756 books had no reviews, and every one of them had an average rating of zero. All 901 records with missing category paths also had zero recorded category memberships.

2.3 Applying data cleaning decisions

I created a separate clean_books DataFrame so that the original data stayed untouched. Missing category paths were replaced with “Unknown”, and I added a binary category_missing indicator to keep a record of which books originally had no category. This way the missing information was labelled as unavailable, and I did not have to make up a subject category.

For books with no reviews, the average rating of zero was changed to a null value. Their review counts stayed at zero, because having no reviews is a real observation, whereas an average customer rating simply doesn’t exist for these books.

All 20,000 records were kept. I verified the result: there were 901 unavailable-category flags, 5,756 null average ratings and no null or blank category-path fields left.

2.4 Creating the bestseller target and examining class distribution

I created a binary target named ‘bestseller’ using project operational definition:

Target	**Definition**
1 - Bestseller	Amazon sales rank between 1 and 10,000 inclusive
0 - Other book	Amazon sales rank above 10,000

Spark calculated the class counts and percentages, and I displayed them in a bar chart shown in figure 2 below. There were 281 bestsellers, which is about 1.41% of the dataset, compared with 19,719 other books, which is about 98.60%.

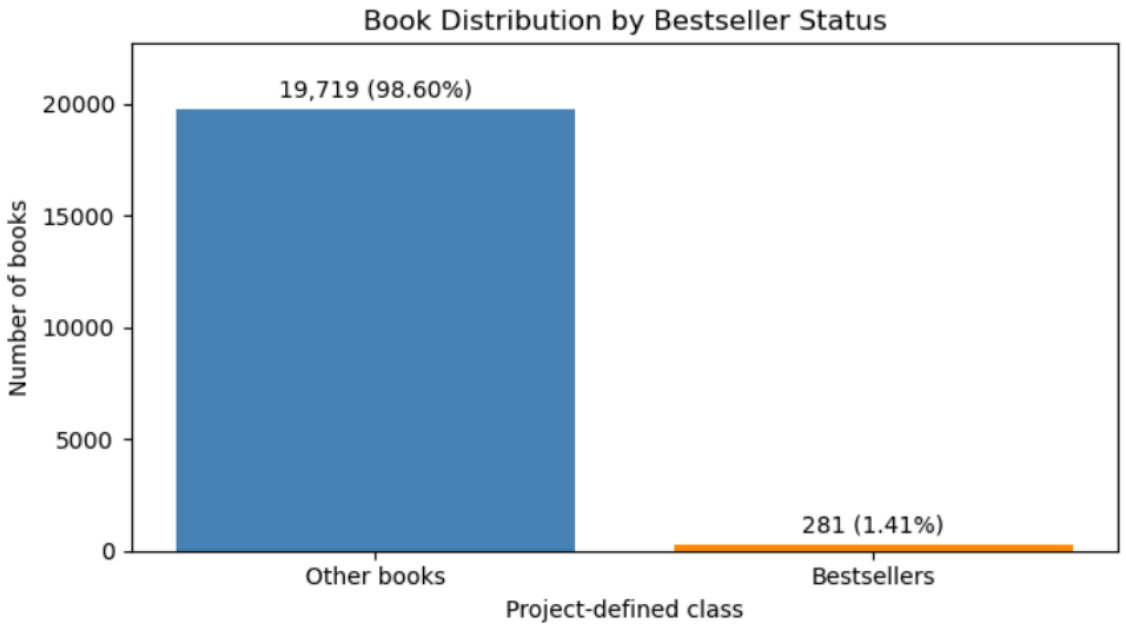


figure 2

This large imbalance showed why accuracy alone would be misleading. If the model simply predicted every book to be an ordinary seller, it would be right about 98.60% of the time and still find no bestsellers at all. For this reason, I put the most weight on precision, recall, F1 and average precision when evaluating the model (Saito and Rehmsmeier, 2015).

2.5 Examining numerical distributions and potential outliers

Spark calculated the counts, means, standard deviations, minimums, quartiles and maximums for sales rank, category count, review count and the available average ratings.

Review counts were heavily right-skewed. The mean was 12.12 reviews, but the median was only two, and the maximum was 4,924. Books had a median of three category memberships. Among the 14,244 books that had an average rating, the mean rating was about 4.32.

When I compared the two classes, bestsellers had a median of 30 reviews, while other books had a median of two. The average ratings, though, were similar: 4.30 for bestsellers and 4.32 for other books. Bestsellers also belonged to more categories on average, at 4.55 compared with 3.61.

I inspected the ten most-reviewed records and did not remove them automatically as outliers. Only three of them met the bestseller definition, which shows that a high number of reviews did not necessarily go with a sales rank in the top 10,000 in this snapshot. A large review count alone was not enough evidence that the data was wrong.

2.6 Visualizing review and category distributions

I grouped the review counts into six ranges: zero, 1–5, 6–20, 21–100, 101–500 and more than 500. Spark calculated how many books fell into each range, and a bar chart showed the results.

In total, 14,603 books, or about 73.02%, had between zero and five reviews. Only 62 books, or 0.31%, had more than 500 reviews. This supports the earlier finding that the distribution is heavily right-skewed.

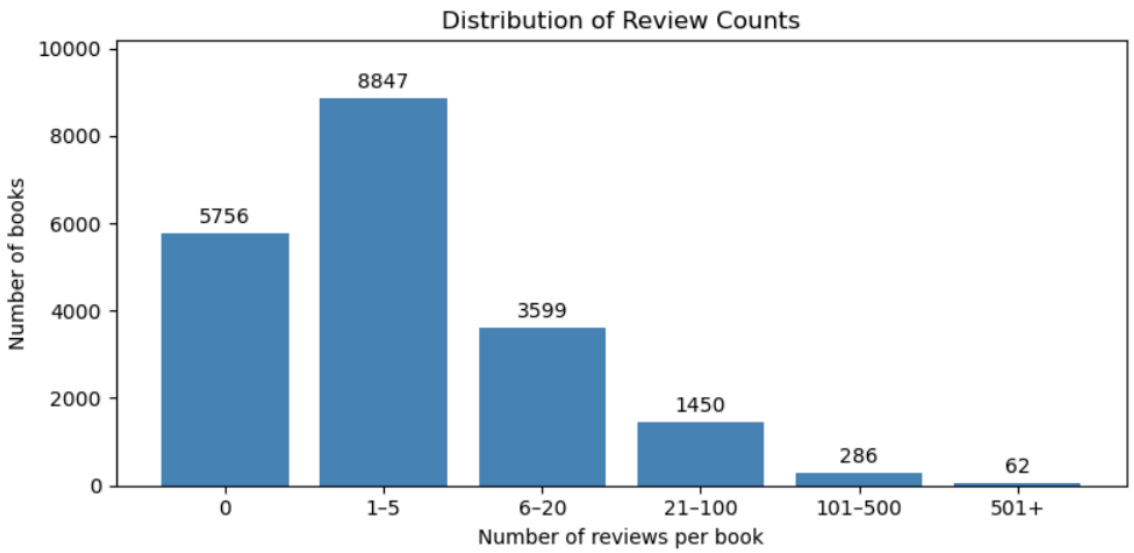


figure 3

A second chart (figure4) showed the category-membership counts. Most books belonged to between two and four categories, and only a few had large counts. This supported the idea of applying a log transformation to the category count before modelling. Books with zero recorded memberships could still be told apart through the missing-category indicator.

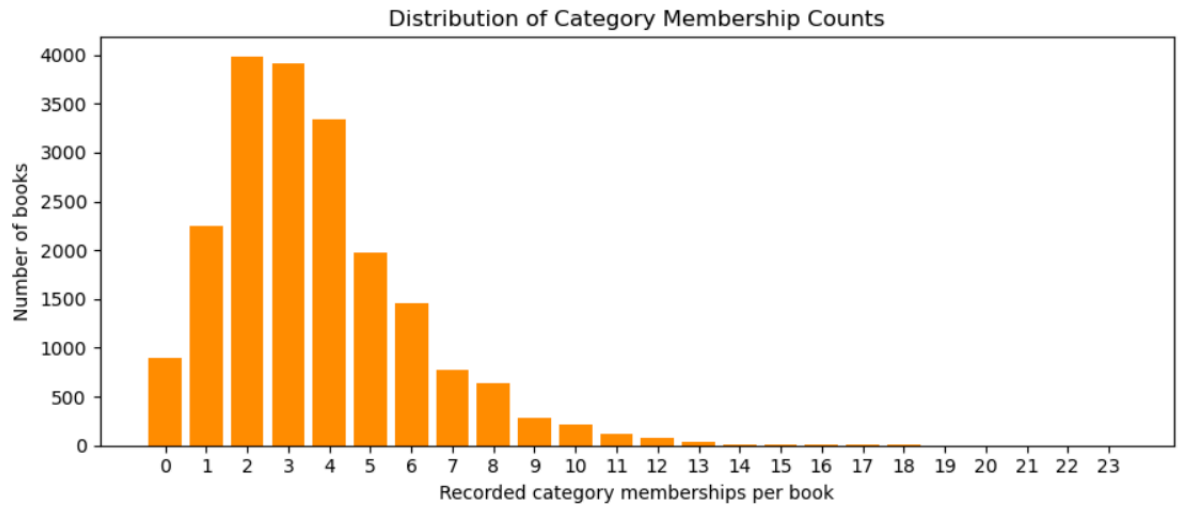


figure 4

2.7 Exploring title characteristics and subject categories

I created two numerical features from the titles: the number of characters and the number of words. Then I compared their summaries across the two classes.

Bestsellers had slightly longer titles on average, at 53.28 characters and 8.50 words, compared with 51.25 characters and 7.88 words for other books. Both classes had a median of seven words, so the title characteristics were broadly similar. These small differences on their own did not show that title length would be a useful predictor.

Next, I pulled out eight subject indicators from the category paths, using the category identifiers recorded in the data. Spark then calculated the number of books, the number of bestsellers and the bestseller percentage for each subject.

Subject	**Bestseller percentage**
Health	3.68%
Business	2.79%
Children	1.79%
Fiction	1.67%
Science	1.44%
Religion	1.07%
History	0.66%
Computers	0.65%

A horizontal bar chart in figure5 compared these percentages with the overall bestseller rate. Health and business had the highest shares, which suggests that the subject indicators may hold useful predictive information. However, the subject groups overlap, and I treated the relationships as associations only, not as proof that a subject causes a book to be commercially successful.

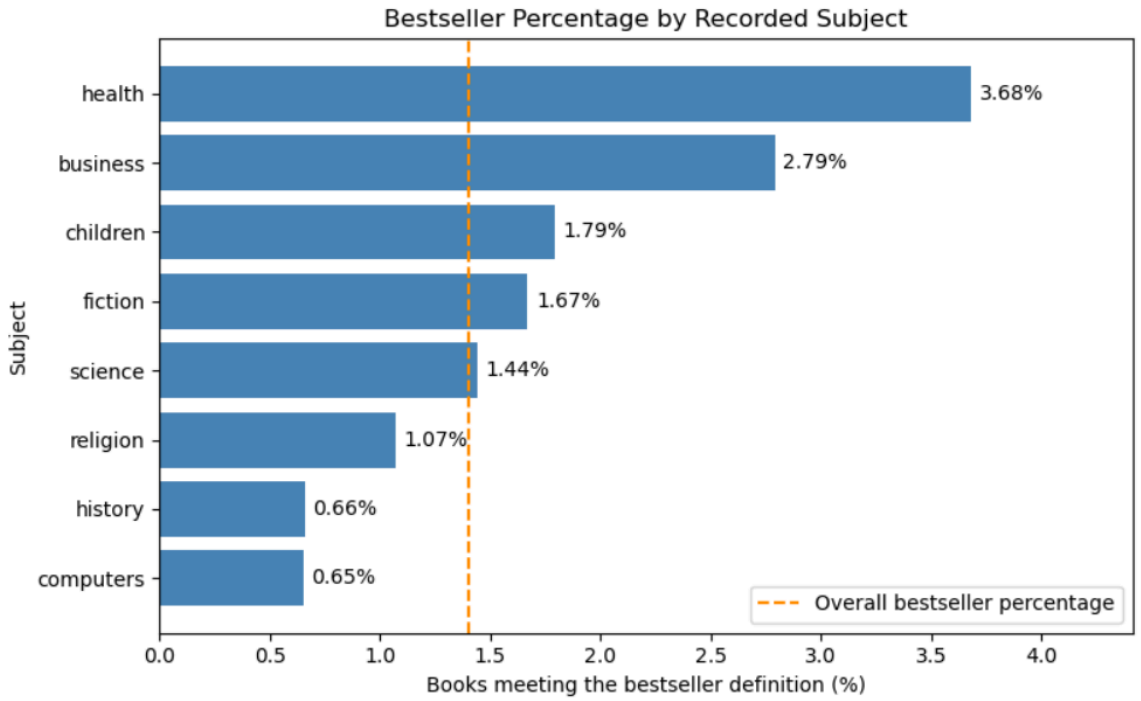


figure 5

3. Feature Selection and Data Preparation

Feature selection combined domain knowledge, predictor-quality checks and correlation filtering. The aim was to retain interpretable variables while reducing redundancy and preventing information leakage.

3.1 Identifying candidate predictors

I started with twelve candidate predictors:

- Title character count and title word count.
- The log-transformed category-membership count.
- A missing-category indicator.
- Eight subject indicators: fiction, children, business, religion, history, science, health and computers.

Each subject indicator showed whether that category appeared in a book's category paths. The missing-category indicator separated books with no category information from books with a recorded subject. I transformed the category count using `log(1 + category_count)`, which compresses its right-skewed distribution.

I left out sales rank because it directly defines the target. I also left out the ASIN, the exact title identity and the product identifiers. I kept review counts and ratings for the EDA but kept them out of the main model, since they would not be available for books that haven't been published yet, and they could simply reflect sales that had already happened. Leaving out information that wouldn't be available when the prediction is made helps avoid misleading evaluation results (James et al., 2023).

3.2 Partitioning before statistical feature selection

Before I calculated any feature-selection statistics, I split the records into training, validation and test sets. Books with identical normalised titles were kept in the same partition, which lowers the risk of leakage between different editions of the same book.

Partition	**Records**	**Bestsellers**
Training	11,997	156
Validation	3,997	50
Test	4,006	75

I did the split with 'GroupShuffleSplit', using fixed random seeds so the results can be reproduced. I checked that every record was assigned to exactly one partition and that no normalised title appeared in more than one of them. From then on, all the feature-selection and scaling statistics were calculated from the training data only. This follows the principle that preprocessing should never learn from the evaluation data (scikit-learn developers, n.d.).

3.3 Checking predictor completeness and variation

I checked the training predictors for missing values, infinite values and the number of distinct values. All twelve predictors were complete and finite, so no imputation was needed.

Every predictor had more than one distinct value, so none of them was removed for being constant. The binary subject and missing-category indicators contained both zeros and ones, which confirms that they vary within the training set.

3.4 Removing highly correlated predictors

I calculated a Pearson correlation matrix using the training observations. The rule I set in advance was to remove a predictor if its absolute correlation with a predictor I had already kept was above 0.90.

Title word count and character count had a correlation of about 0.947, so they overlap a lot. Following the feature order I had declared, I removed ‘title_words’ and kept ‘title_chars’. No other pair went over the threshold.

This cut down on redundant information, but it did not show that the predictors I kept are statistically significant or the best possible choice for prediction. I used correlation filtering and not p-value elimination, because the study is about predictive modelling and not significance testing (James et al., 2023).

3.5 Final feature set

Eleven predictors remained:

Feature Group	**Selected Predictors**
Title characteristics	title_chars
Category information	Log-categories, category_missing
Subject indicators	Fiction, children, business, religion, history, science, health, computers

I kept the missing-category flag so that books with no category information would not be treated the same as books whose categories are known but fall outside the selected subjects.

3.6 Standardising the selected predictors

I standardised the selected predictors by subtracting each one’s training mean and dividing by its training standard deviation:

$$x_{scaled} = \frac{x - \mu_{training}}{\sigma_{training}}$$

The same parameters were then applied, unchanged, to the validation and test records. This put the predictors on similar scales, which helps gradient descent, and it also stopped information from the evaluation data from leaking in through the scaling (James et al., 2023; scikit-learn developers, n.d.).

I checked the result, and the training means were approximately zero and the standard deviations were one. The binary indicators were standardised as well.

Finally, I added a column of ones for the intercept, which gave 12 model-input columns: eleven predictors and one intercept. The predictor and target arrays had matching record counts in every partition.

Code attribution: Group-based partitioning used scikit-learn's `GroupShuffleSplit`; correlation calculations used pandas; array conversion and standardisation used NumPy (Harris et al., 2020). No prebuilt classifier was used during feature selection or preparation.

4. Model Implementation and Training

4.1 Implementing Logistic Regression from scratch

I built the logistic regression in NumPy and did not use a ready-made classification model. The implementation calculated a linear score from the eleven prepared predictors and an intercept. It then applied the sigmoid function to turn that score into an estimated probability of the book being a bestseller. This follows the standard logistic regression formulation described by James et al. (2023).

The code included functions for calculating probabilities, the binary cross-entropy loss, the gradients, the coefficient updates and the class predictions. NumPy handled the underlying array and matrix operations (Harris et al., 2020).

4.2 Defining the training objective

The model worked by minimising binary cross-entropy, which measures how far the predicted probabilities are from the actual outcomes. I also included an optional L2 penalty to discourage large coefficients, and the intercept was left out of this penalty.

Training used full-batch gradient descent, so each update was calculated from all 11,997 training observations. The coefficients started at zero and were adjusted step by step in the direction that lowered the objective. I used numerically stable versions of the sigmoid and the loss calculations to avoid exponential overflow (James et al., 2023).

4.3 Checking the implementation

Before training, I compared the analytical gradient with a numerical gradient calculated using finite differences. The largest absolute difference was about 1.36×10^{-10}, which supports the gradient calculation being correct for the example I tested.

I also tested the sigmoid function with extreme inputs, and it returned finite probabilities that made sense. These checks show that the implementation behaves correctly, but they don't tell us anything about how well the model predicts.

4.4 Training the initial model

The initial configuration used the following settings:

Setting	**Initial value**
Learning rate	0.1
Maximum gradient updates	600
L2 penalty	0
Initial coefficient	zero
Early-stopping tolerance	Loss change below 10^-8 for 10 successive updates

The training loss went down from 0.6931 to 0.0709. The loss curve dropped quickly at first and then levelled off, with no obvious oscillation or divergence. Training did stop at the 600-update limit, though, and not because it met the early-stopping tolerance, so I can't say the model had converged. Figure 6 shows the training loss.

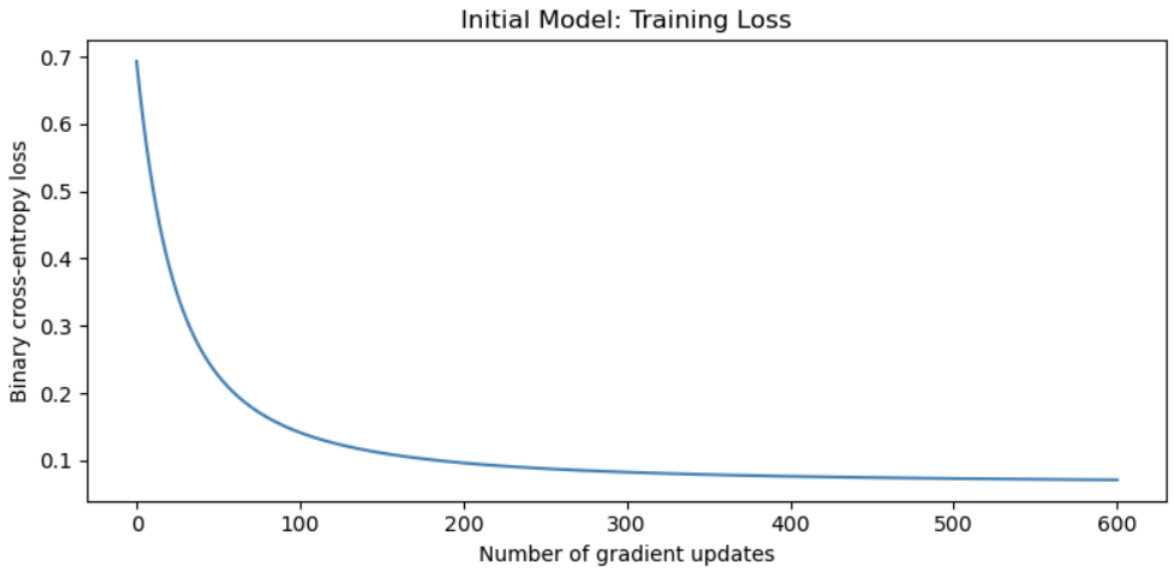


figure 6

4.5 Assessing initial validation performance

I evaluated the trained model on the validation set and compared it with a baseline that gave every book the bestseller prevalence found in the training set.

At the default threshold of 0.5, the initial model predicted no bestsellers at all. Its accuracy of 98.75% was therefore the same as the majority-class baseline, while its recall and F1 were both zero. Its average precision was 0.0239, compared with the baseline's 0.0125, which suggests the model holds some information for ranking books.

I gave average precision the most weight because bestsellers are the rare class. Precision–recall evaluation tells us about how well the model finds the minority class, which accuracy alone can hide (Saito and Rehmsmeier, 2015).

4.6 Retraining with alternative parameters

Three alternatives were trained using the same training data:

Configuration	**Learning rate**	**L2 penalty**	**Validation average precision**	**Validation log loss**
Initial	0.1	0	0.0239	0.0702
Alternative 1	0.3	0	0.0250	0.0661
Alternative 2	0.1	0.01	0.0236	0.0702
Alternative3	0.3	0.01	0.0239	0.0662

I tried a larger learning rate to see if the model would optimise faster, and I tried L2 regularisation to see if limiting the size of the coefficients would improve validation performance. All the configurations ran the full 600 updates.

I selected Alternative 1, which used a learning rate of 0.3 and no L2 penalty, because it had the highest validation average precision. Validation log loss was the tie-breaker I had set in advance. I did not use the test set to choose the configuration.

4.7 Selecting the classification threshold

The selected model still did not predict any bestsellers at a threshold of 0.5. So I looked at its validation probabilities across a range of candidate thresholds and chose the one that gave the highest validation F1.

The threshold I chose was about 0.04061. At this threshold, the validation precision was 7.78%, recall was 14% and F1 was 0.10. The model found seven bestsellers among the 90 books it flagged.

Choosing a threshold changed the rule for classifying books, but it did not require retraining the coefficients. I then fixed both the model and the threshold, at full precision, before running the final test evaluation. Keeping development and testing separate in this way helped keep the assessment of the finished model independent.

5. Model Interpretation and Evaluation

5.1 Selecting appropriate evaluate metrics

I used several complementary metrics because only 281 of the 20,000 books met the bestseller definition. With so few positives, accuracy alone could give a misleading picture of how well the model performs. Precision–recall measures are especially useful when evaluating imbalanced classification problems (Saito and Rehmsmeier, 2015).

Metric	**Purpose in this study**
Average precision	Primary measure of ranking performance across recall levels.
Precision	Proportion of flagged books that actually met the bestseller definition.
Recall	Proportion of actual bestsellers identified.
F1-Score	Summary balancing precision and recall.
Balanced accuracy	Average of sensitivity and specificity, giving both classes equal importance.
ROC-AUC	Assessment of the model's ability to rank positives above negatives.
Log loss	Assessment of predicted probabilities, penalising confident incorrect predictions.
Confusion matrix	Counts of correct predictions and classification errors.

The metrics were calculated using scikit-learn functions, while the classifier itself remained the NumPy implementation developed from scratch (scikit-learn developers, n.d.).

5.2 Establishing a baseline

The baseline gave every book the bestseller prevalence calculated from the training set. At a threshold of 0.5, this meant it classified every book as a non-bestseller.

This gave me a reference point for checking whether the model offered any information beyond simply predicting the majority class. On the test set, the baseline reached 98.13% accuracy but did not detect a single bestseller. Its average precision was 0.0187, its ROC-AUC was 0.5000 and its log loss was 0.0941.

5.3 Using validation data for modeling decisions

I used the validation data to compare the four training configurations. The model with a learning rate of 0.3 and no L2 penalty was selected because it had the highest validation average precision.

I then chose a threshold of about 0.04061 by maximising validation F1. At this threshold, the model found 7 of the 50 validation bestsellers and produced 83 false positives. Precision was 7.78%, recall was 14% and F1 was 0.10.

These are development results and not an independent final assessment, because the validation data was used to choose both the configuration and the threshold. I fixed the model and the threshold before looking at how it performed on the test set.

5.4 Evaluating the frozen model on test data

Final evaluation used the **4,006 test records**, including **75 bestsellers**. Neither the coefficients nor the threshold was adjusted using these outcomes.

Metric	**Baseline**	**Selected model at frozen threshold**
Accuracy	98.13%	96.23%
Balance accuracy	0.5000	0.5230
Precision	0.00%	5.81%
Recall	0.00%	6.67%
F1-score	0.0000	0.0621
Average precision	0.0187	0.0446
ROC-AUC	0.5000	0.6888
Log loss	0.0941	0.0895

The model did better than the baseline on average precision, ROC-AUC and log loss, which suggests it holds some predictive information. However, its low precision, recall and F1 showed that it was not very effective at picking out individual bestsellers.

Accuracy went down at the tuned threshold because the model started making positive predictions, and that brought in false positives. This shows why overall accuracy was not enough to judge whether the model meets the business goal.

5.5 Interpreting the confusion matrix

The final confusion matrix showed:

Outcome	Number	Business Interpretation
True negatives	3,850	Other books are correctly classified.
False positives	81	Books incorrectly flagged, potentially leading to wasted promotional effort.
False negatives	70	Bestsellers missed, representing potential lost opportunities.
True positives	5	Bestsellers correctly identified.

Only **5 of the 86 flagged books** were actual bestsellers, while **70 of 75 bestsellers were missed**. Consequently, the model was considered unsuitable as a standalone basis for publishing or marketing decisions.

5.6 Examining the precision recall curve

I plotted the test precision–recall curve together with a line showing the test prevalence for reference. The average precision of 0.0446 was higher than the baseline of 0.0187, but precision stayed low across most recall levels.

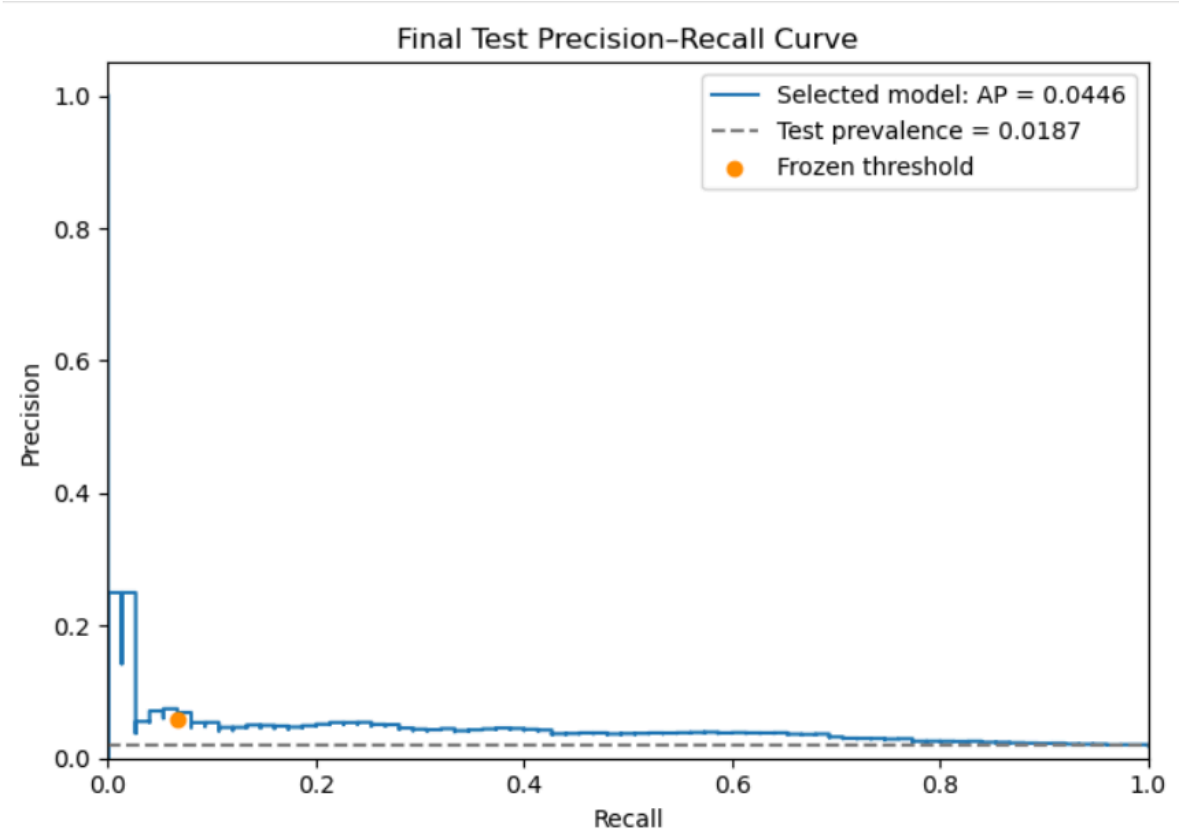


figure7

I marked the frozen threshold on the curve so that the ranking performance could be linked to the actual classification decision. Changing the threshold shifted precision and recall, but it did not change the underlying probabilities, the average precision or the ROC-AUC. The curve supported the conclusion that the model holds some information for ranking books but is of limited use for actually detecting bestsellers (Saito and Rehmsmeier, 2015).

5.7 Interpreting the coefficients

I looked at the fitted coefficients to explain the direction of the links between the predictors and the estimated odds of a book being a bestseller. A positive coefficient meant higher estimated odds, and a negative coefficient meant lower estimated odds, with the other included predictors held constant (James et al., 2023).

Because the predictors were standardised, the exponentiated coefficients first showed odds multipliers for an increase of one training standard deviation. For the binary indicators, I also converted the coefficients into odds ratios for a change from zero to one.

Books recorded under health were associated with about 2.84 times the estimated odds of being a bestseller, and books under business with about 1.73 times the odds. Books recorded under computers were associated with lower odds, with an odds ratio of about 0.48. Title character count had a coefficient close to zero.

These are fitted associations. They are not causal effects, and they are not evidence of statistical significance. In particular, they do not show that changing a book's subject would make it commercially successful.

5.8 Assessing reliability and limitations

Several things support the credibility of the evaluation. Books with identical normalised titles were kept together, the preprocessing statistics were calculated from the training data only, the implementation was checked numerically, and the test data was held back until all the development decisions were fixed. I made no further changes to the model after looking at the final test results.

There are some weaknesses too. The evaluation used a single grouped split, with only 75 positives in the test set and no confidence intervals. The share of bestsellers also differed between the validation and test sets, so differences between their metrics can't be put down to the model's behaviour alone.

Finally, the historical snapshot had no predictors confirmed to be available before publication and no later sales outcomes. The findings therefore show how well books can be classified by their past sales rank, and they do not confirm that the model can forecast future bestsellers.

6. Conclusion

In this project, I built logistic regression from scratch in NumPy, with Apache Spark helping to analyse and prepare 20,000 book records. Retraining the model and choosing the threshold on the validation set improved how many bestsellers were detected, compared with using the default threshold.

The final model reached a test average precision of 0.0446 and a ROC-AUC of 0.6888, which beats the baseline on the ranking measures. However, it found only 5 of the 75 bestsellers, so it is of limited practical use.

The model should therefore stay an academic prototype and should not be used on its own to guide publishing decisions. To assess whether future bestsellers can be predicted, we would need current data, richer predictors that are available before publication, and time-based validation. The present findings apply only to historical classification under this project's sales-rank definition.

References

Apache Software Foundation (2024) *Spark SQL, DataFrames and Datasets guide: Spark 3.5.3*. Available at: Spark SQL and DataFrames - Spark 3.5.3 Documentation (Accessed: 18 October 2026)

Harris, C.R. et al. (2020) 'Array programming with NumPy', *Nature*, 585, pp. 357–362. Available at: https://doi.org/10.1038/s41586-020-2649-2. (Accessed: 18 October 2026)

James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J. (2023) *An Introduction to Statistical Learning: With Applications in Python*. Cham: Springer. Available at: An Introduction to Statistical Learning (Accessed: 18 October 2026)

Leskovec, J., Adamic, L.A. and Huberman, B.A. (2007) 'The dynamics of viral marketing', *ACM Transactions on the Web*, 1(1), Article 5, pp. 1–39. Available at: https://doi.org/10.1145/1232722.1232727. (Accessed: 18 October 2026)

Saito, T. and Rehmsmeier, M. (2015) 'The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets', *PLOS ONE*, 10(3), e0118432. Available at: https://doi.org/10.1371/journal.pone.0118432. (Accessed: 18 October 2026)

scikit-learn developers (n.d.a) *Common pitfalls and recommended practices*. Available at: 12. Common pitfalls and recommended practices — scikit-learn 1.9.1 documentation (Accessed: 18 October 2026)

scikit-learn developers (n.d.b) *GroupShuffleSplit*. Available at: 3.4. Metrics and scoring: quantifying the quality of predictions — scikit-learn 1.9.1 documentation (Accessed: 18 October 2026)

scikit-learn developers (n.d.c) *Metrics and scoring: quantifying the quality of predictions*. Available at: 3.4. Metrics and scoring: quantifying the quality of predictions — scikit-learn 1.9.1 documentation (Accessed: 18 October 2026)

Stanford SNAP (n.d.) *Amazon product co-purchasing network metadata* [Dataset; collected summer 2006]. Available at: SNAP: Network datasets: Amazon co-purchasing network metadata (Accessed: 18 October 2026)